

Semantic Search Overview

Semantic Governance

Semantic Search Overview

Semantic mapping by Metaphactory can help improve the reliability and precision when matching terms, acronyms, symbolics, and alternate labels. Semantic search is only a subset of the capabilities within Search as a Service.

Search & Discovery: Federated Graph Query (basics thru advanced search)

- Token-Based
- Word-Based (includes stemming)
- Symbolic (logic, taxonomy, ontology)
- Boolean (If/Then, AND, OR, NOT)
- Phrase (exact sequence of words)
- Query (structured, unstructured, conditional, constraints)
- Augmented (incorporates persona-based relevancy, history, ranking, contextual relevancy)
- Intelligent (dynamic personalization and forced ranking, proactive filtering, learned behavior)
- Solution-Oriented (problem solving, advanced pattern matching, coupling deductive and inductive reasoning)

Search as a Service (SaaS) goes well beyond traditional token (keyword-based) retrieval toward intelligent, context-aware search capabilities. Asking a question (Query Based Search) requires reverse engineering the string of words, mapping the words to specific vocabulary terms within domains may require inference rules and then deriving constraints to filter the results. This is where [Semantic Engineering](#) plays a key role and is only the first step since the query needs to be applied to downstream repositories (data stores, databases, data lakes) where the conditional logic and constraints can be applied for extracting the results to respond back to the original query.

Semantic search provides the underlayment for building the above scaffold of capabilities. Metaphactory is a key part of the solution.

Search as a Service

Semantic Governance

Search as a Service (Breakdown)

Problem and Scope

Objective:

Enable users to query multiple domain-specific knowledge graphs (finance, party, persona, sales, network monitoring) through a unified interface.

Challenges:

- Heterogeneous ontologies and data structures.
- Distributed data sources (data lakes, knowledge graphs).
- Mapping user inputs (UX parameters) to underlying ontologies and data.

Key Components

Knowledge Graphs (KGs)

- Finance KG: Tracks financial transactions, accounts, and metrics.
- Party KG: Represents entities (e.g., individuals, organizations) and their relationships.
- Persona KG: Captures user profiles, preferences, and behaviors.
- Sales KG: Manages sales data, customer interactions, and revenue.
- Network Monitoring KG: Tracks network performance, devices, and incidents.

Ontologies

- Finance Ontology: Concepts like Account, Transaction, Balance.
- Party Ontology: Concepts like Person, Organization, Role.
- Persona Ontology: Concepts like User, Preference, Behavior.
- Sales Ontology: Concepts like Customer, Order, Revenue.
- Network Monitoring Ontology: Concepts like Device, Incident, Performance.

Data Sources (500 Tables for DIS Ingestion: EDW focus)

Large-scale storage for structured, semi-structured, and unstructured data.

Data is mapped to the ontologies in the KGs.

Semantic Governance

Search as a Service (Breakdown)

Search & Discovery: Federated Query [\(see flow & design\)](#)

A system that integrates and queries distributed KGs.

Examples: Neo4j, GraphDB, DataPlex, other solutions using SPARQL / CYPHER.

User Interface (UX)

A screen where users input query parameters.

Parameters are mapped to ontology concepts and data structures.

Mapping UX Parameters to Ontologies

The UX screen allows users to input query parameters (e.g., "Find all transactions for a customer with network issues"). These parameters must be mapped to the underlying ontologies and data structures.

Example UX Parameters:

- Customer ID: Maps to Party:Person and Sales:Customer.
- Transaction Date: Maps to Finance:Transaction.
- Network Issue: Maps to NetworkMonitoring:Incident.

Mapping Process (simplistic):

- Parameter Extraction: Extract user inputs from the UX screen.
- Ontology Mapping: Map inputs to concepts in the relevant ontologies.
 - 🟡 Example: "Customer ID" → Party:Person and Sales:Customer.
- Data Structure Mapping: Map ontology concepts to data structures in the data lake.
 - 🟡 Example: Party:Person → customer_table in the data lake.

Semantic Governance

Search as a Service (Breakdown)

Federated Query Flow

The federated query engine integrates data from multiple KGs and data lakes. Here's how it works:

Query Decomposition

Break down the user query into subqueries for each KG.

Example: "Find all transactions for a customer with network issues" → Subqueries for Finance, Party, Sales, and NetworkMonitoring KGs.

Query Translation

Translate subqueries into the query language of each KG (e.g., SPARQL for RDF-based KGs, SQL for relational data).

Example: Finance:Transaction → SPARQL query on the Finance KG.

Query Execution

Execute subqueries on the respective KGs and data lakes.

Use federated query engines like Apache Jena or Stardog to handle distributed queries.

Result Aggregation

Combine results from subqueries into a unified response.

Example: Merge transaction data, customer details, and network incident reports.

Example: Federated Query

User Query:

"Find all transactions for customer C123 with network issues in the last 30 days."

Query Breakdown:

- Party KG: Retrieve Party:Person details for C123.
- Sales KG: Retrieve Sales:Customer details for C123.
- Finance KG: Retrieve Finance:Transaction details for C123
- Time KG: Retrieve Time: Map temporal boundaries for records that were created in the last 30 days.
- Network Monitoring KG: Retrieve NetworkMonitoring:Incident details for C123.

SPARQL Query Generator:

Note: This is very simplistic since our actual KGs may be much more complex and we will need to accommodate pathing down sub-domains as we build out the complexity. We also need to align the where clause to the patterns we established for Relations.

Semantic Governance

Search as a Service (Breakdown)

Key Challenge: We will not be storing content (rows if you will) in the persistent knowledge graphs.

Party KG

```
SELECT ?person WHERE {  
  ?person a Party:Person .  
  ?person Party:hasID "C123" .  
}
```

Sales KG

```
SELECT ?customer WHERE {  
  ?customer a Sales:Customer .  
  ?customer Sales:hasID "C123" .  
}
```

Finance KG

```
SELECT ?transaction WHERE {  
  ?transaction a Finance:Transaction .  
  ?transaction Finance:involvesCustomer "C123" .  
  ?transaction Finance:date ?date .  
  FILTER (?date > "2023-09-01"^^xsd:date)  
}
```

Network Monitoring KG

```
SELECT ?incident WHERE {  
  ?incident a NetworkMonitoring:Incident .  
  ?incident NetworkMonitoring:affectsCustomer "C123" .  
}
```

Result Aggregation:

Combine results from all subqueries into a unified JSON or table format.

Search & Discovery (Flow)

Semantic Governance

Search & Discovery (Flow)

- **Scope (order of operations)**
 - Capture the Prompt & Analyze it
 - Domains (short list)
 - Problem Space (Question)
 - Persona, Location, Time, Event
 - Technical Input (aka Project ID)
 - Refinement / Clarification Process
 - Find the IRI for the Unifying Ontology
 - Search across Domain-Specific Ontologies and find the short list of domains
 - Generate a Federated SPARQL Query across the Top-Down Domains to get a match to a pre-defined domain (in the prompt)
 - Generate a SPARQL Query against the specified domain ontology and get a match on the columns using the facets from the prompt
 - Filter the list of columns using constraints inferred from the prompt
 - Generate a package of results:
 - GCP Project ID
 - GCP Database Name(s)
 - GCP Table Name(s)
 - GCP Column Name(s)
 - Preserve the RI using the Container Classes
 - Grouped by Project ID
 - Grouped by Database Name
 - Grouped by Table Name
 - Validate the packaging & groupings
 - Send the package down to “Neo4j Orchestrator”
 - Apply transforms from the packages
 - Generate CIPHER queries

Semantic Governance

Search & Discovery (Flow)

- Submit to Cloud Spanner

• Design

- Prompt Engineering
 - Prompt Analysis (Reverse Engineer: domain relevancy)
 - Semantic Engineering (TBD)
- Knowledge Engineering
 - Extending Knowledge
 - Mapping Top-Down to Bottom-Up
- Vector DB setup & design
 - Contextual Framing
 - Relevancy Mapping: Scenarios
 - Training (pos & neg)

• Challenges

- Monitoring & Measurement
 - Feedback
 - Capture audit info as triples
 - Store triples in Graph
- Learning
 - Incorporate Feedback into Training & Validation
- Training
 - Positive Reinforcement
 - Prompt Analysis
 - Context Engineering
 - Search Top-Down
 - Search Mappings to Bottom-Up
 - Filtering using Constraints
 - Packaging
 - Negative Reinforcement
 - Prompt Analysis

Semantic Governance

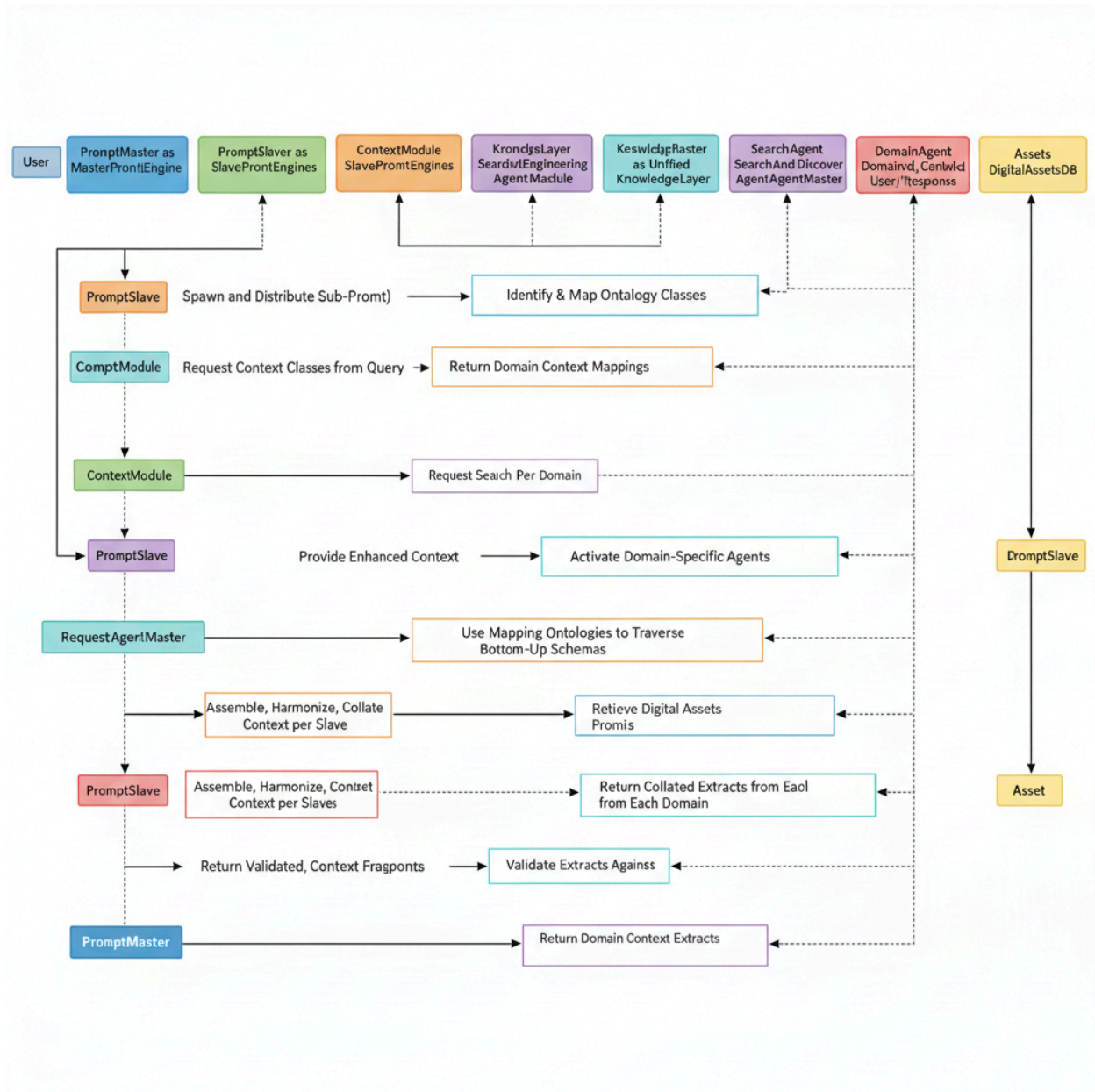
Search & Discovery (Flow)

- Context Engineering
- Search Top-Down
- Search Mappings to Bottom-Up
- Filtering using Constraints
- Packaging

Search & Discovery (Design)

Semantic Governance

Search & Discovery: Sequence Diagram + Orchestration



- The main prompt spawns sub-prompts handled in parallel, each supplied with precise context from the context engineering module.
- Search agents use mapped ontologies and bottom-up schema traversals to extract relevant digital assets across domains.
- Extracts are collated, harmonized, and validated against the original prompt before final assembly and delivery to the user.

Semantic Governance

Search & Discovery: Sequence Diagram + Orchestration

Orchestration Agent

- **Central Coordinator:** Acts as the primary controller, receiving user queries and directing the workflow, ensuring each specialized agent (master prompt, slave prompts, domain-specific search agents) operates in the correct sequence and under the right context.
- **Context-Driven Task Routing:** Integrates tightly with the context engineering module to shape the context for each agent/subagent using the latest retrieved, structured, and harmonized knowledge.
- **State and Memory Management:** Maintains the flow of context and task state, ensuring each agent works with up-to-date information and that outputs are aggregated and validated before reaching the user.
- **Dynamic Adaptation:** Can reassign or re-sequence tasks for subagents based on intermediate results, agent confidence scores, or external events, optimizing for relevance and completeness of the final response.

Orchestration Flow (with Agentic Fabric)

- **User Query Intake:**
 - 🟡 Orchestration agent receives and parses the user's input.
- **Master Prompt Initialization:**
 - 🟡 Creates a main prompt strategy, shaped by global context from the context engineering module.
- **Subtask Decomposition:**
 - 🟡 Splits the overall intent into parallel or sequential subprompts and tasks.
 - 🟡 Allocates slave prompts to specific subtasks (each shaped by relevant contextual class or mapping).
- **Domain-Specific Agent Activation:**
 - 🟡 Invokes domain-specific agents, providing each with context classes and schema mappings from the unified knowledge layer as identified by the context engineering module.
- **Parallelized Search & Extraction:**
 - 🟡 Orchestration agent oversees extraction as each domain agent traverses ontologies and schemas, collects assets, and returns structured context fragments.
- **Assembly & Harmonization:**
 - 🟡 Consolidates and harmonizes the extracted content from all agents.
 - 🟡 May trigger additional prompt iterations or clarifications if outputs are incomplete or inconsistent.

Semantic Governance

Search & Discovery: Sequence Diagram + Orchestration

- Validation Loop:
 - 🟡 Compares aggregate results to the original master prompt and user intent.
 - 🟡 Ensures relevance, completeness, and compliance with the original requirements.
- User Response Delivery:
 - 🟡 Delivers the consolidated, validated, and context-rich output back to the user, maintaining transparency and traceability through the entire agentic workflow.